front, the same walk as `getIndex`, and its cost grows with the length of the list.

```
Item List::removeLast() {
    // find the penultimate node
    Node* tmp;
    for (tmp = myFirst; tmp->myNext->myNext == nullptr; tmp =
tmp->myNext);

    delete myLast;
    myLast = tmp;
    myLast->myNext = nullptr;
    mySize--;
}
```

The loop and the return value are where the hour ended. The tests do not pass yet.

Part 5. Close

> **Indexing walks, `removeFirst` costs the same at any length, and `removeLast` has to walk to the end.**
>
> **Monday:** `removeLast` finished, and the tests passing.

CS112: Friday 23 October, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM W06-LISTS AND ERIC'S ACCOUNT OF THE CLASS

Before the break we wrote the List in `w06-lists`. Node lives inside it, one `delete myFirst` frees the whole chain, and `append` and `prepend` grow it from either end, with a test case for each.

Today: reaching into the middle of the list, and taking nodes out. `getIndex`, `operator[]`, `removeFirst` and `removeLast`, with tests for the first three.

PART
1. `getIndex` walks
2. `operator[]` borrows it
3. `removeFirst`
4. `removeLast`
5. Close

Part 1. `getIndex` walks

> **Talk to your neighbor.** `v[3]` on a `vector` costs the same as `v[0]`. What does `l[3]` cost on our List, and what about `l[3000]`?

There is no base address to add to. Item 3 is wherever node 3 happens to be, and the only way to find node 3 is to start at the first and follow `myNext` three times.

```
Item& List::getIndex(unsigned ix) const {
    if (mySize == 0) {
        throw invalid_argument("List is empty, nothing to index
into\n");
    }
    if (ix < 0 || ix > mySize-1) {
        throw out_of_range("You are going out of limits!\n");
    }
    Node* tmp = myFirst;
    for (int i = 0; i < ix; i++) {
        tmp = tmp->myNext;
    }
    return tmp->myItem;
}
```

The cost grows with the index. `l[3000]` takes a thousand times as many steps as `l[3]`.

Part 2. `operator[]` borrows it

```
Item& List::operator[](unsigned ix) {
    return getIndex(ix);
}
```

One line, so the bounds check is written once and both ways of indexing get it. Both return `Item&`, which is what lets an index sit on the left of an assignment.

```
TEST_CASE("indexing", "[list]") {
    List l;
    l.append(3);  // 0
    l.append(4);  // 1
    l.append(5);  // 2
    l.append(15); // 3

    REQUIRE(l[0] == 3);
    REQUIRE(l.getIndex(2) == 5);
    REQUIRE(l[3] == 15);

    l[1] = 40;
    REQUIRE(l[1] == 40);

    REQUIRE_THROWS_AS(l[4], out_of_range);
}
```

Part 3. `removeFirst`

> **Talk to your neighbor.** The first node has to go, and its item comes back to the caller. Which pointers move, and in what order?

`myFirst` moves on to the second node, and the old first node is deleted.

```
Item List::removeFirst() {
    if (mySize == 0) throw underflow_error("List is empty you
fool!\n");
    Node* tmp = myFirst;
    myFirst = myFirst->myNext;
    tmp->myNext = nullptr;
    Item it = tmp->myItem;
    delete tmp;
    mySize--;
    return it;
}
```

> **Talk to your neighbor.** What goes wrong if the line `tmp->myNext = nullptr;` is left out?

Wednesday's chain reaction. `delete tmp` runs Node's destructor, which deletes `tmp->myNext`, and that is the node `myFirst` now points to. The whole rest of the list goes with it. Cutting the link first is what makes the delete free exactly one node.

The test empties a three-item list one call at a time, checks that a fourth call throws, and then appends again to show an emptied list still works.

Part 4. `removeLast`

> **Talk to your neighbor.** `removeFirst` never looks past the second node. Which node does `removeLast` need to find, and why can `myLast` not hand it over?

The second to last. It becomes the new last node, and its `myNext` has to be set to `nullptr`. `myLast` points the other way: a node knows the one after it, never the one before. So `removeLast` walks from the